

6: Statistik. Brug af Procedure.

Programmet i fig. 6.1 optæller et observationssæt $x_1, x_2, x_1, \dots, x_N$ og repræsenterer det ved et stolpediagram. Stolperne består af små kasser, som programmet stabler op efterhånden som optællingen skrider frem. Det konkrete eksempel er 200 tællinger fra en Geiger-tæller (8, 10, 3, 5, ...), der tæller i ét sekund ad gangen på en kilde, der forbliver lige aktiv i hele observationsperioden. Det efterfølgende program - se fig. 6.2 - er en udbygning af det første, og undersøger om fordelingen er den forventede en Poissonfordeling. Hertil grupperer det observationerne i intervaller og udregner de tilsvarende intervallfrekvenser.

KLAR

```
Proc Kvadrat(x,y)
  Line(x-0.5,y,x+0.5,y)
  Line(x+0.5,y,x+0.5,y+1)
  Line(x+0.5,y+1,x-0.5,y+1)
  Line(x-0.5,y+1,x-0.5,y)
endproc
SelectWindow(Graf1)
ClearLines

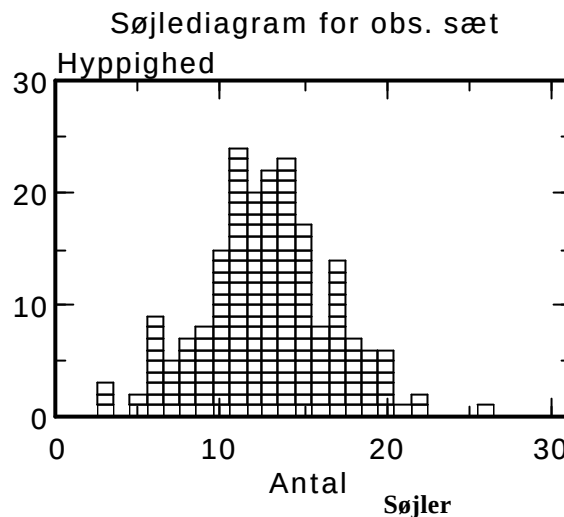
For i:= 0 to Int(maxn*1.5) do
  Hyph[i]:=0
Next
i:=0
```

LØKKE

```
Kvadrat( n[i], Hyph[ n[i] ] )
Hyph[ n[i] ]:=Hyph[
n[i] ]+1
i:=i+1
Pause(vent)

If i>N-1 then stop
```

← Sættes vent lig f.eks. 1, så indlægges her en pause på 1 sekund. Så vil kvadraterne i søjlediagrammet blive stablet op i samme fart som de kom ud da geigertælleren målte.



	1	2
	n	Hyp
0	8	0
1	10	0
2	3	0
3	5	3
...	...	...

Figur 6.1: Program *MAT6a.FPR*. En Geiger-tæller har N ($= 200$) gange registreret et antal radioaktive henfald indenfor 1 sekund. Disse N tællinger er indlæst i søjlen med navn n ($n[0]= 8$, $n[1]= 10$, $n[2]= 3$, ..., $n[199]= 12$). Navnet *Hyph* står for *Hyppighed*, og søjlen udfyldes i programmets LØKKEdel, således at *Hyph[i]* kommer til at angive det antal gange, tallet i forekommer i søjlen n .

KLAR

Først formuleres proceduren *Kvadrat(x,y)*. En procedure er et underprogram, der bringes til udførelse blot ved at skrive dets navn. Hvis man senere skriver f. eks. *Kvadrat(3,8)*, så tegnes et kvadrat med kantlængde 1 med punktet (3,8) midt på bundlinjen. Den næste ordre, *SelectWindow(Graf1)*, fortæller hvilket grafvindue der skal tegnes i - sjovt nok¹ er den nødvendig her selvom kun eet grafvindue er åbent. *Clearlines* sletter alle de streger, der blev tegnet under foregående simulation. *For - to - do* - løkken nulstiller hyppighederne i søjlen *Hyph*.

LØKKE

arbejder sig een ned gennem gennem n -søjlen i hvert gennemløb. Mødes for eks. tallet 7, så placeres et kvadrat ovenpå den hyppighedssøjle, der i de foregående gennemløb er opbygget over $x = 7$.

¹ Faktisk er den kun nødvendig første gang programmet kører efter at have været lukket ned. Ved gentagelse kan den udelades. Gad vide om dette har været programmørens hensigt?

Bemærk den såkaldte *Procedure* der indleder *Klar*-delen af programmet i fig. 7.2. En procedure er et underprogram, der bringes til udførelse, blot man senere i programmet nævner dens navn. Den her anvendte procedure *Kvadrat(x,y)* tegner et kvadrat med kantlængden 1 således, at det kommer til at stå med bundlinjens midtpunkt i punktet (x,y) . Procedurens handling er således udtrykt ved de to parametre x og y , og man siger derfor, at den er *parameterafhængig*, eller *parametriseret*. Det er lige meget hvad man kalder parametrene. De er *lokale* på samme måde som den variabel x , vi brugte i *MAT3b.fpr* til at udtrykke regneforskriften $f(x) = 0,2x^2 - 6$ for funktionen f (jvf. Bemærkning 2 til figur 3.2.) . Vi husker, at hvis $f(x) = 0,2x^2 - 6$ "forkortes" til blot $f = 0,2x^2 - 6$, så vil et kald af f give $f(x)$ med den værdi x har i det øjeblik funktionen kaldes. Programmet kan så ikke forstå meningen med eks. $f(3)$. Det forholder sig på helt tilsvarende måde med procedurer, hvis handling afhænger af parametre. Endvidere kan det også ved procedurer forekomme, at det er bekvemt at indføre nogle ekstra variable mellem *Proc* og *Endproc*. Hedder disse f. eks. *antal*, x og $Q3$, så kan vi gøre disse lokale, ved at skrive *Local antal, x, Q3* i procedures første linje. Et eksempel herpå er givet i fig. 6.3 nedenfor, der beskriver en procedure, der tegner en cirkel i stedet for et kvadrat.

Vi vil nu udbygge programmet i fig 6.1, således at det også kan vurdere om den fundne fordeling svarer til den teoretisk forventede Poisson-fordeling. Iflg. denne skal sandsynligheden for n henfald kunne skrives som

$$P(n) = \frac{\mu^n}{n!} e^{-\mu} \quad (6.1)$$

hvor μ er det gennemsnitlige antal, bedømt ud fra en *meget* lang række af observationer. Det kan vises, at denne fordeling har spredningen $\sigma = \sqrt{\mu}$, samt at ca.67 % af sandsynligheden skal ligge i intervallet $[\mu - \sigma, \mu + \sigma]$ og at ca. 96 % skal ligge i $[\mu - 2\sigma, \mu + 2\sigma]$. Selvom vi kun har 200 observationer, skulle disse formler gerne passe "rimeligt", når vi sætter μ og σ lig middelværdi og standardafvigelse for observationssættet. Jvf. (C16-19) i appendiks C. Programmet er beskrevet i forbindelse med fig. 6.2 nedenfor.

KLAR

{ Proc Kvadrat(x,y) og Hyph = 0 som i fig.6.1}

For i:= 0 to 5 do

 IH[i]:=0

Next

Proc Sort(a)

 If a<=Mn-2*sMn then IH[0]:=IH[0]+1

 If Mn-2*sMn<a AND a<=Mn-sMn then IH[1]:=IH[1]+1

 If Mn-sMn<a AND a<=Mn then IH[2]:=IH[2]+1

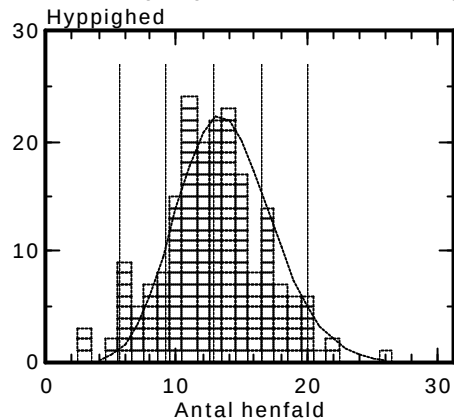
 If Mn<a AND a<=Mn+sMn then IH[3]:=IH[3]+1

 If Mn+sMn<a AND a<=Mn+2*sMn then IH[4]:=IH[4]+1

 If Mn+2*sMn<a then IH[5]:=IH[5]+1

endproc

Sammenligning med Poisson-fordeling

**LØKKE**

If i<N then

 Kvadrat(n[i],Hyph[n[i]])

 Hyph[n[i]]:=Hyph[n[i]]+1

 Sort(n[i])

 i:=i+1

 Pause(vent)

Endif

if i>=N AND r<xmaxthen

 P:=N*(Mn^r/Fac(r))*exp(-Mn)

 r:=r+1

endif

If r>=xmaxthen

 Line(Mn-2*sMn,0,Mn-2*sMn,ymax*0.9)

 Line(Mn- sMn,0,Mn- sMn,ymax*0.9)

 Line(Mn ,0,Mn ,ymax*0.9)

 Line(Mn+ sMn,0,Mn+ sMn,ymax*0.9)

 Line(Mn+2*sMn,0,Mn+2*sMn,ymax*0.9)

 stop

Endif

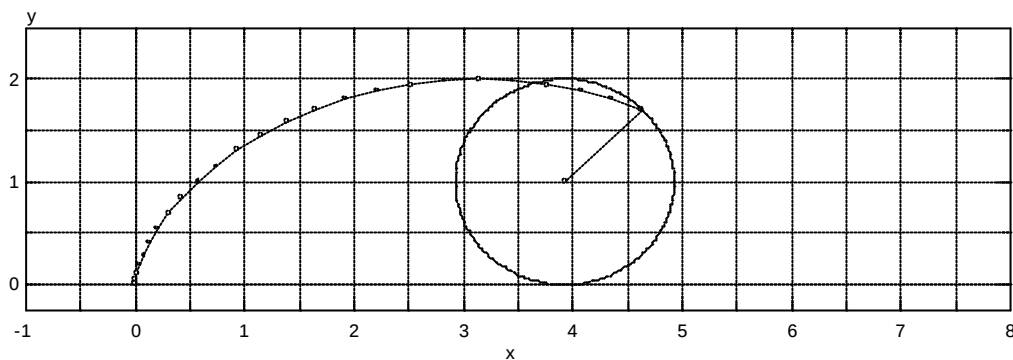
Navn	Udtryk	Værd	Tekst
N	Number(n)	200	Antal gange der er målt
Mn	Mean(n)	12.85	Gennemsnitligt antal tællinger
sMn	sqrt(Mn)	3.585	Kvadratroden af gennemsnittet
	Sdev(n)	3.925	Standardafvigelsen
pm1	(IH[2]+IH[3])*100/N	64.5	% mellem plus-minus kva
pm2	pm1+(IH[1]+IH[4])*100/N	95.5	% mellem plus-minus 2 x kva

	1	2	3
Tekst		Hyppighed	Intv.hyppighed
Navn	n	Hyph	IH
0	8	0	5
1	10	0	29
2	3	0	59
3	5	3	70
4	14	0	33
5	6	2	4
6	6	9	
7	6	5	
8	8	7	

Figur 6.2: Udskrifter fra programmet MAT6b.FPR. Dette starter med at lave samme stolpediagram som i fig. 6.2 Dette sker ved hjælp af den første af løkkedelens 3 If-sætninger. Når denne er fuldført, udføres det der står If-sætning nr. 2: Den forventede Poissonfordelingskurve indtegnes. Herefter går til sidste If-sætning i hvilken de 5 lodrette streger tegnes. Disse markerer observationsintervallerne.

Bemærk at første If-sætning i 4. linje har en ekstra procedure *Sort(n[i])* i forhold til samme programdel i fig. 6.1. Denne sorterer observationerne ud i de relevante intervaller og optæller intervalhyppighederne i søjlevinduets 3. søjle.

Procedure der tegner en cirkel

**Klar**

```

PROC Cirkel(a,b,r)
Local t,x1,x2,y1,y2
t:=0

Repeat
  x1,y1:=a+r*cos(t),b+r*sin(t)
  t:=t+pi/90
  x2,y2:=a+r*cos(t),b+r*sin(t)
  SelectWindow(Graf 1)
  line(x1,y1,x2,y2)
until t>2*pi
ENPROC

SelectWindow(Graf 1)
clearPoints
clearLines

t:=0
Plot(t,1,7) =====>
Cirkel(t,1,1)
x,y:=0,0

```

Løkke

```

t:=t+dt
clearlines
clearpoints

Plot(t,1,7)
Cirkel(t,1,1)

x:=t*cos(6*pi/4-t)
y:=1+sin(6*pi/4-t)
line(t,1,x,y)

if t>2*pi then stop

```

Plot(t,1,7) plotter i punktet (t,1) det symbol, der har koden 7 (en lille cirkel). Se app.E i FPro-vejl.

Figur 6.3. Program *MATX6c.FPR*. Ovenfor indførte vi proceduren *Kvadrat(x,y)*, der tegner et bestemt kvadrat. Nu indføres proceduren *Cirkel(a,b,r)*, der tegner cirklen med centrum i (a,b) og radius r - dvs cirklen med parameterfremstillingen $x(t) = a + r \cos t$ og $y(t) = b + r \sin t$ (Repeat-løkken tegner $2^\circ = \pi/90$ radianer af cirklen i hvert gennemløb). Bemærk den indledende linje: *Local t,x1,x2,y1,y2*, der gør de nævnte variable *lokale* i forhold til proceduren. Dette betyder, at de samme betegnelser frit kan anvendes andre steder i programmet. Variable a, b og r i *Cirkel(a,b,r)* har automatisk denne egenskab. *Løkke*-delen af programmet har blot til formål at demonstrere et par nyttige småting:

FPro's *Løkke* gennemløbes én gang, når man taster *Ctrl+F8* (eller klikker *Afvikling/Kør løkken én gang*). Man opdager herved, at det enkelte gennemløb sletter cirklen, fra det foregående gennemløb, og gentegner den et lille stykke dt ($\approx \pi/20 = 0,157$) længere fremme. Læseren opfordres til at prøve dette og selv at forsøge at forstå, hvorledes man herved får indtryk af et "cykelhjul", der ruller henad x-aksen medens ventilen beskriver den optegnede bane.

Læseren har måske undret sig over den usystematiske brug af store og små bogstaver i *Klar*-delens *clearPoints* og *clearLines*. Formålet er blot at demonstrere, at vi kan vælge, som vi har lyst til, i disse indbyggede standardprocedurer. Den procedure *Cirkel(a,b,r)*, vi selv har defineret, tillader ikke denne frihed (medmindre vi via *Indstillinger / Beregninger* indstiller programmet til generelt ikke at skelne mellem store og små bogstaver)